Development Plan
Software Engineering

Team 19, Team BVSH
Shivya Mehta
Anindita Bornomala
Harmanpreet Singh Sagar
Vanessa Lai

Table 1: Revision History

Date	**Developer(s)**	**Change**
28 September 2026	Shivya Mehta	Workflow Plan, Project Decomposition and Scheduling, Expected Technology, Coding Standard
28 September 2026	Anindita Bornomala	IP to protect, Copyright License, Team Meeting Plan, Team Communication Plan
28 September 2026	Harmanpreet Singh Sagar	POC demonstration plan, Reflection and Team Charter Appendices
28 September 2026	Vanessa Lai	Confidential Information, Team Member Roles, Workflow plan - use of CI/CD, Use of AI/LLMs
...	...	...

This document describes the team's development plan for the CAS Midterm and Assignment Optimization system. The system is designed to collect assessment information from CAS course outlines, identify periods where students may have a high assessment workload, suggest alternative assessment dates to professors, and check course outlines against McMaster's course outline policy and CEAB accreditation requirements.

The document first covers topics related to confidential information, intellectual property, and the copyright license for the project. It then explains how the team plans to work together, including meeting expectations, communication methods, team member roles, and the Git and GitHub workflow that will be used throughout development. The document also outlines the project schedule and task breakdown, the proof of concept plan, the technologies expected to be used, the team's approach to AI and LLMs, and the coding standards that will be followed. The appendices include the team's generative AI use disclosure, a reflection on the deliverable, and the team charter.

1 Confidential Information?

Yes, the project may involve confidential information related to sensitive CAS department data that will be determined later during future supervisor meetings. This information will be kept confidential and will not be shared publicly or uploaded to third-party websites without authorization.

2 IP to Protect

Intellectual Property (IP) transfer for this project is marked as "Maybe", and is still subject to further discussion with the CAS administrative supervisors (Matt Vonk, Connie Carrabs). The team will ensure that any IP-related concerns are addressed appropriately, and in the event that there is IP to protect (ex. due to potential internal administrative workflows), all team members will sign the "Intellectual Property Guide Acknowledgement" and abide by any formal IP agreements established with the CAS supervisors.

3 Copyright License

The team is adopting the MIT License for the open-source base components of the codebase, located in the root directory of our repository in the LICENSE file. The MIT License provides high flexibility for software reuse while granting the CAS Administration team full rights to modify, deploy, and maintain the software indefinitely without restriction.

4 Team Meeting Plan

4.1 Team Internal Meetings

- **Frequency & Time:** The team will meet weekly on Mondays, Wednesdays, and Thursdays at 5:30 PM (during lecture hours), as well as Wednesdays from 2:30 PM to 4:30 PM (tutorial hours). All other meetings will be scheduled as needed, with a minimum 24-hour notice to all team members.

- **Location:** All in-person meetings will take place either in lecture or at a nearby campus library study room. Virtual meetings will be conducted via Microsoft Teams.

- **Structure:** Each meeting will have a designated Meeting Chair (will rotate weekly) to enforce the agenda and a Notetaker to keep track of meeting minutes and action items. Meetings will follow a structured agenda consisting of status updates, GitHub Kanban board review, roadblock discussions, and assigning action items post-meeting.

4.2 Supervisor/TA Advisor Meetings

- **Frequency & Time:** The team will meet bi-weekly for about 30 to 60 minutes with CAS supervisors (Matt Vonk, Connie Carrabs), with each meeting being scheduled within 24 hours of the previous meeting. Additional meetings may be scheduled as needed, but this is the current plan to ensure consistent communication and feedback with the supervisors, while considering their availability. The team will also engage in 20-minute meetings with the TA advisor (Lucas Dutton) on Wednesdays, ranging from 2:30 PM to 4:30 PM (tutorial hours) to discuss project progress, roadblocks, and ask questions related to the project. All TA meetings have been scheduled in advance, as per the Software Engineering Capstone calendar (typically bi-weekly, with exceptions for events such as Proof-of-Concept demos).

- **Location:** In-person meetings with the CAS supervisors will take place at a location convenient to the supervisors (likely a meeting room booked in advance). Meetings with the TA advisor will be held in-person at ITB 120. If a team member or advisor cannot attend in-person, a hybrid meeting option can be arranged (case-by-case basis). All virtual meetings will be conducted via Microsoft Teams.

- **Structure:** The team will distribute a meeting agenda prior to each supervisor meeting. The Chair will lead the presentation and discuss recent project milestones and progress, while the Notetaker documents meeting minutes and feedback from the supervisors/advisor.

5 Team Communication Plan

- **Asynchronous Communication:** The team will use Discord as their primary day-to-day communication channel for quick questions, adjustments and review requests, and general project discussions. Asynchronous communication with supervisors and the TA advisor will be conducted via email, with a response time of 24 hours during weekdays.
- **GitHub Issues & Kanban Project Board:** All project tasks, bugs, and feature additions will be tracked using GitHub Issues. Team members will create issues on the CAS Course Optimization Project Kanban Board for unassigned tasks prior to working on them. Issues will be tagged with labels (ex. `documentation`), and linked to the relevant pull requests.
- **Code Reviews & Pull Requests:** Formal communication regarding code changes will occur within GitHub Pull Requests. As each pull requires passing CI checks and at least one review approval from another team member, this will provide the opportunity to provide feedback and ensure code quality before merging the changes into the main branch.
- **Documentation & File Sharing:** Draft deliverables and meeting notes will be maintained using Google Docs, while official source code and documentation will be kept within the primary GitHub repository.

6 Team Member Roles

The team will rotate roles at each milestone to ensure that all members remain engaged across different aspects of the development process and gain a breadth of experience. All team members will contribute to documentation and meeting notes, actively participate in GitHub reviews, and decision-making. Role assignments will be revisited at each milestone to accommodate changing priorities, and ensure that all team members gain experience across different areas of the project.

The roles we anticipate during our project include:

- **Communication Lead**: Leads communication with stakeholders, prepares meeting agendas and notes for TA and supervisor meetings, records decisions during meetings, and creates action items for team members. This role requires strong communication, organization, and collaboration skills to ensure that stakeholder feedback is accurately communicated to the team.
- **Research & Requirements Lead**: Reviews relevant peer-reviewed research, analyzes project requirements, and establishes the team's weekly priorities. This role requires requires skills in research, requirements analysis, and technical writing, as well as an understanding of the academic policies and accreditation requirements relevant to the CAS department.

- **Technical Lead**: Supports architectural and implementation decisions and helps the team debug significant technical issues and communicates with TAs when additional assistance is needed. This role requires software engineering expertise in areas such as system design, database management, software architecture, and development workflows. The Technical Lead also helps ensure that technical decisions align with project requirements and maintainability goals.

- **UI/UX & Product Lead**: Tests weekly product iterations, collects feedback, and focuses on human-computer interaction (HCI) and user experience. This role requires knowledge of user centered design, interface design, usability testing and understanding the needs of the users. This role is related to the course SFWRENG 4HC3, which all team members are taking currently.

The team brings a diverse range of technical and domain expertise to the project, including software architecture, databases, full-stack development, machine learning, software testing, and human-computer interaction. This overlap allows team members to support one another and take on responsibilities beyond their assigned roles when needed.

Summary of the team's existing domain expertise:

Team Members	**Domain Expertise**
Shivya Mehta	Databases, System Design, APIs, Fullstack, Software Requirements, Git, Documentation, HCI
Harmanpreet Singh Sagar	Software Testing, Backend, Git, Databases, Software Requirements, Hardware, Documentation, HCI
Vanessa Lai	Software Architecture, Machine Learning/NLP, Fullstack, DevOps, UI/UX, Software Requirements, API, Documentation, HCI
Anindita Bornomala	Machine Learning, AI, Git, APIs, Fullstack, Software Requirements, Documentation, Software Testing, HCI

Table 2: Team Members and Domain Expertise

7 Workflow Plan

The team will use GitHub for version control, issue tracking, project planning, and code review. The `main` branch will contain stable, production-ready code, while new features and fixes will be developed on separate branches. Each branch will address a specific GitHub Issue and will be merged into `main` through a pull request (PR). The PR will describe the changes, identify the linked issue, and summarize the testing that was performed. Before the PR is merged, at least one team member other than the author must review and approve it.

Branch Type	Format	Example
Feature	`feature/<description>`	`feature/course-upload`
Bug Fix	`fix/<description>`	`fix/calendar-filter`
Refactor	`refactor/<description>`	`refactor/api-structure`
Documentation	`docs/<description>`	`docs/setup-guide`

Table 3: Git Branch Naming Convention

GitHub Issues will be used to track features, bugs, questions, and technical tasks. Issue templates will be created for common tasks such as feature requests and bug reports. Issues will be assigned to team members and linked to relevant milestones.

GitHub Labels such as `feature`, `bug`, `documentation`, `frontend`, `backend`, and `AI/ML` will be used to categorize and organize issues. Priority labels such as `high`, `medium`, and `low` will be used to indicate the urgency of each issue.

GitHub Projects will be used as a project board to track issues through `Backlog`, `Ready`, `In Progress`, `In Review`, and `Done` statuses. The board will be reviewed during team meetings to identify blocked or overdue work, manage dependencies, and adjust priorities.

Milestones will be used to group issues according to major project phases and deliverables, allowing the team to track progress and identify outstanding work. For example, for this project milestones will likely include `MVP`, covering course-outline data extraction and policy checking, and `Final Release`, covering assessment conflict identification, scheduling suggestions, database exports, testing, and documentation.

Continuous Integration/Deployment: GitHub Actions will be used for CI to automatically run tests (unit, integration, acceptance and regression), linting, and formatting checks on pull requests. Changes merged into `main` will be deployed automatically where practical. We will also monitor deployment results, address any failures, and use stakeholder feedback and testing results to guide future development iterations.

Development Workflow: `Issue` → `Feature Branch` → `Implementation` → `Pull Request` → `CI Checks` → `Code Review` → `Merge` → `Deployment`.

8 Project Decomposition and Scheduling

The team will use **GitHub Projects** as the primary tool for scheduling and tracking project progress. A Kanban board has been created and will be used to organize tasks into *Backlog*, *Ready*, *In Progress*, *In Review*, and *Done*. Tasks will be created as GitHub Issues and assigned to individual team members. The

board will be reviewed regularly during team meetings to identify blocked tasks, adjust priorities, and ensure progress toward upcoming deliverables.

Large features will be decomposed into smaller, specific tasks that can be completed and validated within a few hours.

Work will be divided based on team members' skills, interests, and current workload. Team members will aim to spend approximately 10 hours per week on the project, including scheduled class time. Tasks will be distributed to balance the workload and avoid assigning too many dependent tasks to one team member. Team members will also collaborate on tasks that require multiple areas of expertise, such as integrating the frontend, backend, database, and LLM components.

The project schedule will be organized around the major course deliverables and project milestones. Only the first 9 weeks of the project are shown in detail, as the remaining weeks will be planned based on the progress made during the first half of the project. The schedule is as follows:

- **Week 4:** Problem Statement, POC Plan, and Development Plan.
  - Problem Statement, POC Plan, and Development Plan
  - Research text extraction and LLM approaches
  - GitHub repository and GitHub Projects setup
- **Week 5:**
  - Define initial system requirements and supported input formats
  - Confirm that the initial implementation supports PDF course outlines only
  - Document the conversion workflow for Word documents that must be processed
  - Collect sample PDF course outlines for development and testing
  - Evaluate PDF text extraction libraries
  - Define CEAB requirements to be checked by the system
- **Week 6:** Requirements Document Revision 0 and Hazard Analysis.
  - Finalize requirements for PDF course-outline processing and CEAB verification
  - Identify risks related to incorrect text extraction and LLM-generated results
  - Implement initial PDF text extraction prototype
  - Document direct Word document extraction as future work, including evaluation of DOCX extraction libraries or PDF conversion tools

- **Week 7:**
  - Implement initial LLM-based course information extraction
  - Define structured format for extracted course information
  - Design CEAB verification logic
- **Week 8:** V&V Plan Revision 0.
  - Develop tests for text extraction and CEAB verification
  - Define metrics for evaluating extraction accuracy
  - Create initial backend API for course outline processing
- **Week 9:**
  - Integrate text extraction with the LLM pipeline
  - Implement CEAB requirement verification
  - Store extracted course information in database
- **Week 10:** Design Document Revision -1 and CD-Fall Revision 0.
- **Weeks 11–12:** Proof of Concept demonstration and interviews.
- **Week 14:** MVP should be completed and ready for demonstration.
  - Demo MVP results and address remaining issues
  - Plan next-stage features based on stakeholder feedback
- **Week 16:** Design Document Revision 0.
- **Weeks 18–19:** Revision 0 demonstration and interviews.
- **Week 22:** V&V Report Revision 0 and CD-Winter Revision 0.
- **Week 24:** Final Demonstration (Revision 1).
- **Week 26:** EXPO Demonstration.
- **Final Submission:** Final documentation, including the source code and revised project documents.

The GitHub Project is available at: GitHub Project Link.

9 Proof of Concept Demonstration Plan

The main risk for this project is extracting reliable information from course outlines. CAS course outlines do not follow one standard format. Different instructors use different layouts and file types, and some information may be missing or incomplete, such as "Midterm Date: TBD". Since most of the system depends on this information, including the shared calendar, conflict detection, scheduling suggestions, and compliance checking, the extracted data needs to be accurate. If assessment dates, weights, or learning outcomes are extracted incorrectly, the rest of the system will also produce unreliable results.

Another major risk is turning the scheduling problem into something that can actually be solved. The team plans to use the CP-SAT solver from Google OR-Tools. The difficult part will be converting things such as assessment dates, student cohorts, exam periods, and policy restrictions into constraints that the solver can work with.

There are also a few smaller risks. LLM-generated explanations may not always be accurate, and the model may eventually need to be hosted on CAS servers instead of using an external API. In addition, McMaster's course outline policy and CEAB requirements are written for people to interpret, so the team will need to convert them into clear rules that the system can check automatically.

To test whether these risks are manageable, the proof of concept will use real course outlines from the 2025-2026 academic year. The team will focus on courses taken by one cohort, such as second-year Software Engineering students, so that realistic assessment conflicts can be identified. The system will extract assessment names, dates, weights, and learning outcomes from the outlines and store them in a database. Any missing or unclear information will be flagged instead of being ignored. The extracted results will be compared with a manually prepared answer key to measure accuracy. If fully automated extraction is not accurate enough, the team will use a semi-automated approach where the extracted information is pre-filled and then confirmed or corrected by a user.

The extracted assessment data will then be shown in a simple calendar that can be filtered by course. The system will identify at least one week when students in the selected cohort have multiple assessments due close together. For one of these assessments, the solver will suggest alternative dates that reduce the workload while still following important constraints, such as avoiding the exam period, or changing lab due dates. The proof of concept will first use a reduced CP-SAT model containing the most important constraints. If the complete constraint model is too complex, a simpler scoring method may be used only as a fallback to demonstrate the scheduling workflow. In that case, the unresolved CP-SAT constraints will be documented and evaluated before the full system is implemented.

Each suggested date will also include a short explanation generated by an LLM. For the proof of concept, the team plans to use a hosted LLM API. The explanation will only use information produced by the scheduling system, such as comparing how many assessments are scheduled in the original week versus the suggested week. This makes the explanation easier to check against the actual data. Only publicly available course outline information will be sent to the API. The LLM component will also be designed so that it can later be replaced by a model hosted on CAS servers without requiring major changes to the rest of the system.

The proof of concept will also include a basic compliance checker. The sample course outlines will be checked against a small set of rules from McMaster's course outline policy, such as whether required sections are included and whether assessment weights add up to 100%. It will also demonstrate one representative CEAB graduate-attribute check. Any missing or incorrect information will be listed for the user; broader CEAB coverage will be evaluated after the proof of concept.

The user interface will be kept simple for the proof of concept. The primary focus is reliable data extraction and demonstrating alternative assessment dates. The goal at this stage is not to build the final product, but to show that the data pipeline and scheduling workflow work together. The basic policy and representative CEAB checks, along with explanations and other features, will remain limited demonstrations or follow-up work so that the main technical risks can be evaluated within the available time.

If the proof of concept does not address the main risks as expected, the team will use the results to identify what needs to be changed before moving forward. We will review which parts were unsuccessful, determine the reasons, and discuss possible next steps with our supervisors. Depending on the results, we may need to reduce the project scope, such as supporting fewer course outline formats, adding more manual confirmation for extracted data, or moving features like compliance checking and LLM-generated explanations to the stretch goals. The project plan and schedule would then be adjusted so that the team can focus on building a smaller, reliable system rather than trying to complete too many features that may not work well.

10 Expected Technology

This list outlines the technologies we currently expect to use for the project. These choices may change as development progresses.

- **React.js + TypeScript** → Used to build the frontend and provide type safety.
- **Vite** → Frontend build tool and development server for the React application.

- **Tailwind CSS** → CSS framework for styling the frontend application.
- **Python + FastAPI** → Used to build the backend REST APIs and application logic.
- **MongoDB** → Database for storing course information, assessments, learning objectives, and CEAB-related information.
- **Ruff** → Python linter and formatter following PEP 8 conventions.
- **ESLint + Prettier** → Tools for identifying issues and maintaining consistent formatting in the frontend.
- **Pytest** → Testing framework for backend unit and integration tests.
- **Vitest/Jest + React Testing Library** → Tools for testing React components and frontend functionality.
- **GitHub Actions** → CI tool for automatically running tests, linting, formatting, and coverage checks.
- **Git + GitHub** → Version control and collaborative code management.
- **GitHub Projects** → Kanban board for organizing and tracking project tasks.
- **Visual Studio Code** → IDE for development.
- **Docker** → Used to containerize application components and maintain consistent development environments.
- **Hugging Face Transformers** → Library for loading and running pretrained language models for extracting assessment dates, learning objectives, and other information from course outlines.
- **PyTorch** → Deep learning framework used to run LLM inference and support potential model fine-tuning.
- **vLLM** → LLM inference and serving framework for hosting pretrained models on school servers and exposing them through an API.
- **CUDA** → GPU computing platform used to accelerate LLM inference on NVIDIA GPUs.

11 Use of AI and LLMs

The team plans to use Claude Code (version 2.1.283) and Cursor (Grok 4.5 Fast), as AI-assisted development tools. These tools will support planning, development, documentation, and collaboration throughout the project. Specifically, we will use AI to review project documentation against the checklists in the

capstone project GitHub repository (https://github.com/smiths/capTemplate/tree/main/pdfs/Checklists), suggest codebase folder structures and configuration setups, and assist with debugging issues such as missing dependencies and LaTeX compilation errors. AI will also be used to organize and refine meeting minutes, summarize discussions, and extract key decisions and action items. All final decisions and implementations will remain under the team's control.

The team will independently verify all AI-generated suggestions for accuracy, completeness, consistency, and appropriateness before incorporating them into the project. AI-generated code will undergo human code review before being committed or pushed to the repository. Code will be validated through CI/CD pipelines using automated builds, linting, and unit, integration, and regression testing. Dependencies, configuration changes, and potential security issues will be reviewed carefully.

12 Coding Standard

The project will follow the following established coding standards to maintain consistency, readability, and maintainability across the codebase:

- **Frontend:** TypeScript and React code will follow ESLint and Prettier formatting rules.`camelCase` will be used for variables and functions, while `PascalCase` will be used for React components and types.
- **Backend:** Python code will follow PEP 8. `snake_case` will be used for variables and functions, and `PascalCase` will be used for classes.
- **Database:** MongoDB documents will use consistent field naming and structure. Indexes will be used for frequently queried fields, and database operations will be separated from application logic.
- **General:** Code will use descriptive names and consistent formatting. Comments will be used where necessary to explain non-obvious logic. Unused code and debugging statements will be removed before merging.
- **Testing:** Backend code will be tested using Pytest and frontend code using Vitest or Jest testing library. Core functionality, API endpoints, and important application components will have corresponding tests where appropriate.

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

The team used the following AI tools and how they were used:

- Claude Code (Opus 5) for reviewing and refining the document
- GPT-5.6 Luna
- GitHub Copilot - v1.0.88
- Cursor (Grok 4.5 Fast) for formatting and debugging LaTeX errors

13.2 How and Where Used

- Claude Code (Opus 5) was used to review and refine the document, suggesting improvements to structure and clarity, and assisting with grammar and style. It was used to primarily refine the Proof of Concept Demonstration Plan, and the Appendix - Team Charter sections.
- GPT-5.6 Luna was used to ensure the Expected Technology section was clear and concise, and to help identify any missing technologies that may be relevant to the project. The section was then reviewed and modified by the team to ensure accuracy and completeness.
- GitHub Copilot 1.0.89-7 was used to create a table and fix latex formatting issues in the Workflow Plan section.

13.3 Verification of AI-Generated Content

- Any output by Claude Code (Opus 5) was reviewed thoroughly and modified before its suggestions were added to the document.
- Course materials and deliverable requirements were manually reviewed to ensure that all technical assertions, requirements, and references were accurate and complete.

Appendix — Reflection

Why is it important to create a development plan prior to starting the project?

Our system has several components that depend on each other, including data extraction, the database, conflict detection, scheduling suggestions, and compliance checking. Creating a development plan helps make sure everyone on the team has the same understanding of how these components will work together and what tools and formats will be used. It also helps us divide responsibilities and agree on how we will use Git, track issues, and communicate throughout the project. Having these decisions made early reduces confusion later and allows us to focus the proof of concept on the areas that have the most technical risk.

In your opinion, what are the advantages and disadvantages of using CI/CD?

One of the main advantages of CI/CD is that it can catch problems early by automatically running tests whenever changes are made. This is useful for our project because many parts of the system are connected. For example, a change to the format of the extracted course data could affect the calendar, conflict detection, or scheduling solver. CI/CD can help identify these issues before they are merged into the main branch. It can also help keep the main branch stable and make it easier to maintain a working version of the system throughout development.

The main disadvantage is the additional time required to set up and maintain the CI/CD pipeline. It is also only as useful as the tests included in it. If the test coverage is limited, some problems may still go unnoticed. Certain parts of our project, such as evaluating the quality and accuracy of LLM-generated explanations, may also require manual testing rather than relying only on automated tests.

What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Our group did not have any major disagreements while completing this deliverable. Most decisions were made through discussion and agreement between the team members. One topic that required more discussion was how the LLM should be used in the system. We considered using an external API, hosting an existing model on CAS servers, or potentially using our own locally hosted solution.

Since we have not yet had an official meeting with our supervisors, we do not currently know the exact privacy and hosting requirements for the project. For now, we agreed to use a hosted API for the proof of concept and only

send publicly available course outline information to it. We will also design the LLM component so that it can be replaced with a locally hosted model later if required. We plan to discuss the final hosting approach with our supervisors during our first project meeting.

Appendix — Team Charter

External Goals

Our team has four main goals for this project. First, we want to achieve a strong grade that reflects the amount of effort we put into the project throughout the year. Second, we want to create a complete software system that we can confidently discuss in future job interviews, applications, or graduate school opportunities. Third, we want to build something that is genuinely useful for the CAS department and could continue to be used after the course is finished. Ideally, the system would reduce some of the manual work currently done by administrators and professors while also helping students avoid weeks with too many assessments. Finally, we want to use the project as an opportunity to gain hands-on experience with technologies that are new to most of the team, including LLM integration, constraint optimization using OR-Tools, and full-stack web development.

Attendance

Expectations

All team members are expected to attend the weekly meetings that occur during lecture and tutorial times, and any additional meetings scheduled. Everyone should also attend meetings with our supervisors and teaching assistant whenever possible. Team members should come to meetings prepared to give a short update on their progress and discuss any problems they are facing.

If someone knows that they will be late, need to leave early, or cannot attend a meeting, they should let the rest of the team know on Discord as early as possible. Anyone who misses a meeting is responsible for reviewing the meeting notes and catching up on any decisions or tasks that were discussed.

Acceptable Excuse

Acceptable reasons for missing a meeting or deadline include illness, family emergencies, religious observances, or unavoidable academic conflicts such as a test or exam. Planned commitments, such as job interviews or travel, are also understandable as long as the team is informed ahead of time and any affected work is handled beforehand.

Reasons such as forgetting a meeting, oversleeping, or choosing a non-urgent commitment without informing the team would generally not be considered acceptable. Since everyone on the team has other courses and responsibilities, our main expectation is that members communicate early and honestly when something may affect their availability.

In Case of Emergency

If a team member experiences an emergency that prevents them from attending a meeting or completing a task, they should inform the team on Discord as soon as they are reasonably able to. They will not be expected to provide personal details about the situation.

If possible, the member should push any work they have already completed to GitHub and leave a short update explaining what has been done and what is still remaining. This will make it easier for another team member to continue the work if necessary. The team can then redistribute tasks depending on the situation and how close the next deadline is. If the emergency affects a major course deliverable, the team will contact the teaching assistant or instructor to discuss the situation and determine the next steps.

Accountability and Teamwork

Quality

Each team member is expected to make reasonable progress on their assigned tasks and communicate any blockers before they become a larger problem. For written deliverables, we will aim to have individual sections completed at least two days before the final deadline. This will give the team enough time to review the full document, fix inconsistencies, and make sure everything follows the required format.

For development work, code changes will be submitted through pull requests and reviewed by at least one other team member before being merged. Automated checks should also pass before a change is added to the main branch. If someone is unsure about the quality of their work or is having difficulty completing a task, we would rather they raise the issue early so that the team has time to help.

Tasks will be tracked using GitHub issues with an assigned team member and target date so that everyone can see what is currently being worked on. A commitment means an agreed task, meeting, review, or deliverable with a specific due date. A commitment will be considered missed when it is not completed by the agreed date and the team was not notified beforehand, unless an emergency prevented the member from providing notice. Missing two commitments within a rolling four-week period will be considered repeatedly missing commitments. The first step will be to check in with the member privately and understand whether there is a problem or whether they need help. If the issue continues, the team will discuss it together and agree on how to get the work back on track. If the situation still cannot be resolved within the team, we will involve our teaching assistant.

Attitude

We expect everyone on the team to feel comfortable sharing ideas, asking questions, and speaking up when they are unsure about something. Different opinions and approaches are expected, and everyone should treat each other's ideas with respect. When reviewing documents or pull requests, feedback should focus on the work itself and clearly explain what could be improved and why.

Team members are also expected to be reliable and supportive. This means following through on assigned tasks, communicating when something is going wrong, and helping teammates when they need support. We also expect everyone to communicate professionally with our supervisors, teaching assistant, and each other.

Disagreements are a normal part of working as a team. If a disagreement occurs, the people involved should first try to resolve it through a respectful discussion where everyone has a chance to explain their perspective. If the issue affects the project or cannot be resolved directly, the team will discuss it together and follow the decision-making process outlined below. If a conflict continues or becomes personal, we will involve our teaching assistant for additional support.

Stay on Track

To keep the project on schedule, tasks will be tracked through GitHub issues with an assigned team member and target date. This will make it easier for everyone to see what is being worked on, what still needs to be completed, and whether anything is falling behind. During our weekly meetings, we will review our current tasks, discuss any blockers, and adjust responsibilities if someone has too much work.

Larger deliverables will be broken down into smaller tasks with internal deadlines so that problems can be identified before the final submission date. We will also keep track of things such as meeting attendance, completed issues, pull requests, and contributions to deliverables. The purpose of this is not to compare team members, but to make sure the workload remains reasonably balanced throughout the project. As a baseline, each member will aim to contribute approximately 10 hours per week, including scheduled class time. During weekly planning, the team will compare the estimated hours and deadlines for each member's assigned issues. A weekly workload between 8 and 12 hours will generally be considered balanced, with adjustments made for course deadlines, emergencies, and differences in task complexity. If a member's planned workload falls outside this range, the team will redistribute tasks, pair members on larger tasks, or adjust deadlines where possible.

If a team member starts falling behind or is not contributing consistently, another member will first check in with them privately to understand the situation and see if they need help. If the problem continues, the team will discuss

it together and create a clear plan to get back on track, which may include changing responsibilities or setting a new deadline. If the issue still cannot be resolved, we will involve our teaching assistant and the situation may also be reflected in peer evaluations.

We also want to recognize members who put in additional effort or help the team beyond their assigned responsibilities. This can be as simple as acknowledging their contribution during meetings and making sure their work is recognized by the rest of the team.

Team Building

Since all the team members already know each other, we have a good starting point as a team. However, working together on a year-long capstone project will require more communication and coordination than a typical course assignment. We plan to keep communication open and informal through Discord and make sure everyone feels comfortable asking questions or raising concerns.

We will also take time to acknowledge major milestones and completed deliverables, and may occasionally meet outside of project work, such as going for a meal after a major deadline. At the beginning of meetings, members can briefly mention if they have a particularly busy week or other major commitments. This will help us adjust workloads when possible instead of waiting until someone falls behind.

Where possible, we also want team members to work on different areas of the project throughout the year. This will allow everyone to learn more than one part of the system and avoid having one person become responsible for the same type of task for the entire project.

Decision Making

Our team will try to make important decisions through discussion and consensus. When a decision needs to be made, each team member will have the opportunity to share their opinion and explain their reasoning. The goal will be to choose an option that everyone is comfortable moving forward with, even if it is not everyone's first choice.

If we cannot reach an agreement within a reasonable amount of time, we will use a majority vote. Since the team has four members, there is also the possibility of a tie. In this case, we will give more weight to the team member who is working most closely with that part of the system. For larger decisions that could affect the overall direction of the project, we may instead ask our supervisors or teaching assistant for input.

Decisions that depend on CAS requirements, such as where the LLM should be hosted or how certain department data can be used, will be discussed with

our supervisors before being finalized. Important decisions will also be recorded in the meeting notes so that the team can refer back to them later.